



**INSTITUTO
FEDERAL**
Santa Catarina

Evolução do Desenvolvimento de Software

Engenharia de Software II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



cores

tom escuro	#384254	oxford blue
acento escuro	#9599AE	santas gray
cor principal	#E74146	cinnabar
acento claro	#E2AC5B	equator
tom claro	#F8FAF9	snow drift
	#374252	

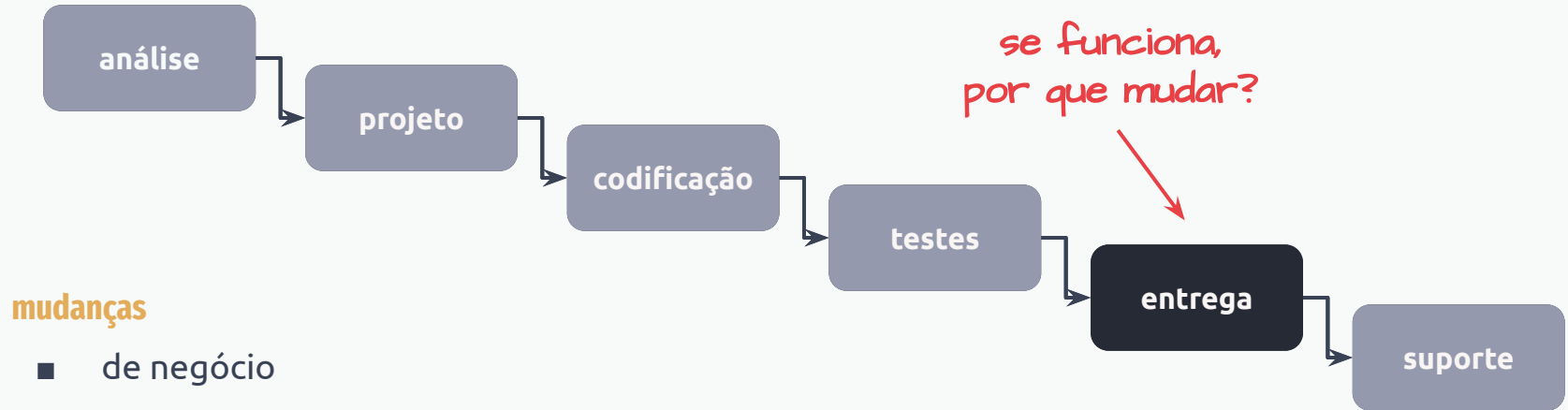
fontes

Fira Sans Extra Condensed

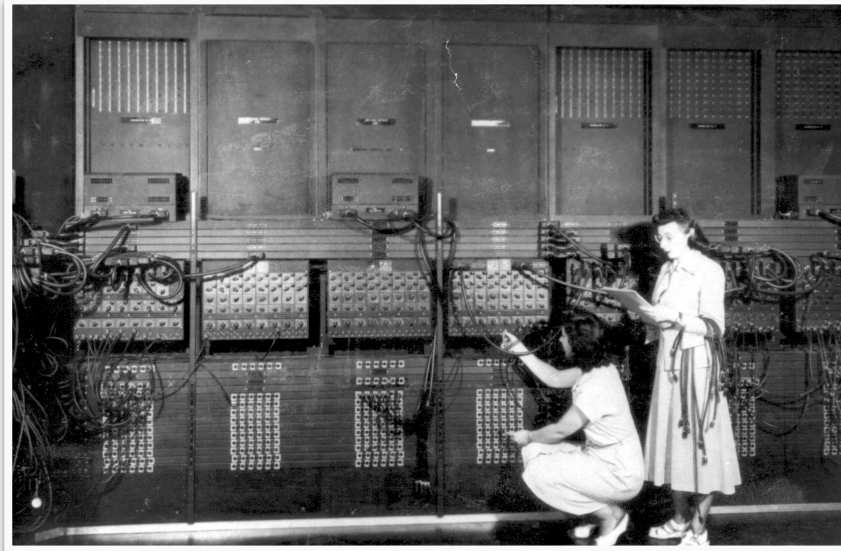
Ubuntu

Roboto Mono

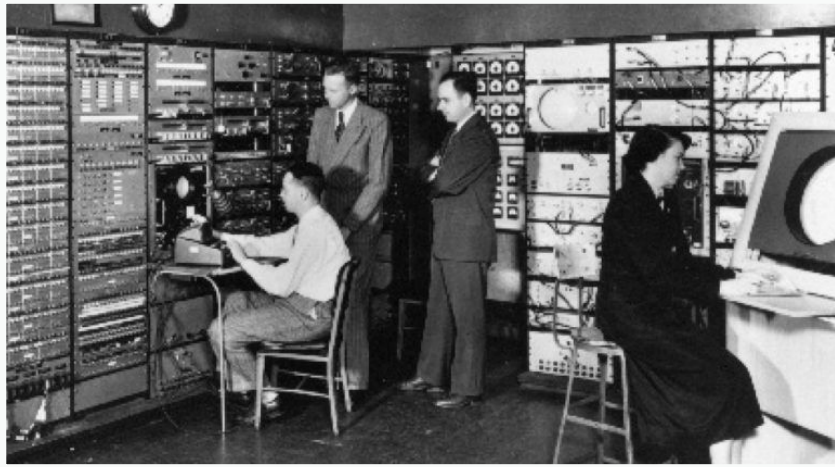
Evolução do Desenvolvimento de Software



Evolução do Desenvolvimento de Software



Evolução do Desenvolvimento de Software



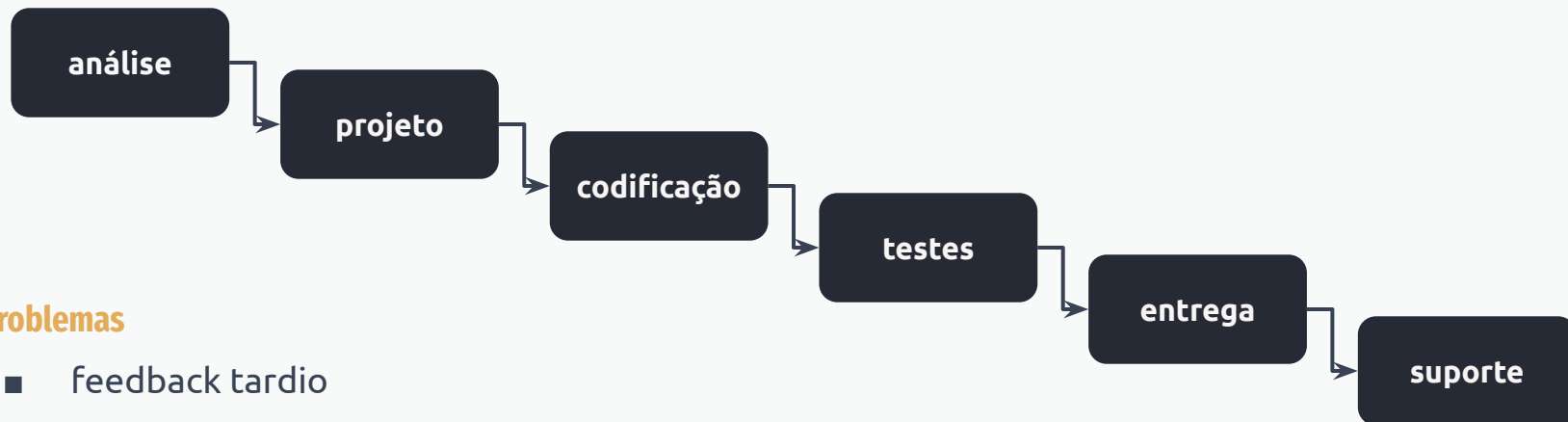
Whirlwind I
computador do projeto SAGE



Conferência de Engenharia de Software da
OTAN (1968)

Evolução do Desenvolvimento de Software

MARCO 1: desenvolvimento sequencial

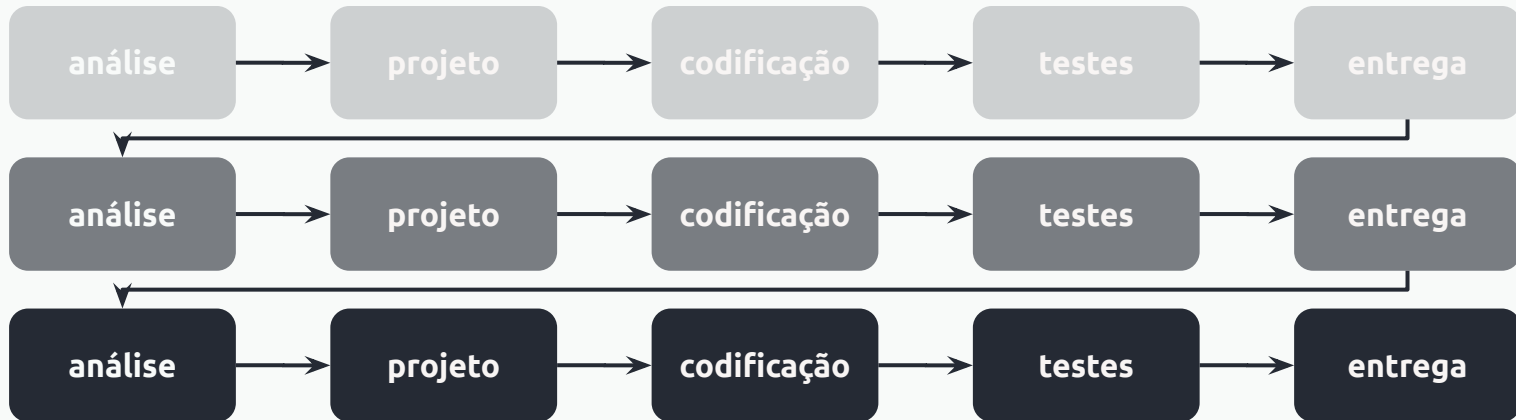


problemas

- feedback tardio
- erro (barato) no início -> erro (caro) no final

Evolução do Desenvolvimento de Software

MARCO 2: desenvolvimento iterativo orientado a risco



reduz a incerteza cedo

Evolução do Desenvolvimento de Software

MARCO 3: abordagem ágil orientada a valor e adaptação

- **indivíduos e interações** > processos e ferramentas
- **software em funcionamento** > documentação abrangente
- **colaboração com o cliente** > negociação de contratos
- **responder mudanças** > seguir um plano

Evolução do Desenvolvimento de Software

MARCO 4: integração frequente e validação automática

- **problema** integração tardia quebra tudo
 - cada pessoa trabalha isolada por muito tempo
 - integração perto da entrega
 - resultado: conflito, regressão e surpresa!
- **resposta** integração contínua
 - integrar mudanças com frequência
 - validar automaticamente
 - tratar falha de *pipeline* como prioridade imediata

cenários

- PR só entra com build e testes verdes e a integração acontece diariamente
- equipe usa pipeline, mas integra branches no fim da sprint
- build falha e o time segue entregando outras coisas por 3 dias

Evolução do Desenvolvimento de Software

MARCO 5: sempre pronto para produzir

- **problema** boa integração não garante bons *releases* com segurança e previsibilidade
 - produção continua sendo evento tenso
 - com checklist manual
 - janela rara
- **resposta** entrega contínua
 - software sempre pronto depois do *pipeline*
- **avanzando** implantação contínua
 - produção automática depois do *pipeline*

exemplo de fluxo de entrega contínua

- *pipeline* cria artefato versionado
- testes passam
- *deploy* em *staging*/homologação pronto
- produção depende de decisão de negócio

Evolução do Desenvolvimento de Software

exercício 1

■ cenários

- **A** PR obrigatório com checks; merge diário; deploy automático em staging; produção só com aprovação de negócio em janela definida
- **B** merge frequente e deploy automático em produção; porém, 40% dos deploys exigem hotfix nas primeiras 2 horas
- **C** build e testes rodam a cada commit; equipe mantém feature branch por 3 semanas antes de integrar
- **D** artefatos versionados e reproduzíveis; homologação sempre atualizada; produção pode ser liberada em minutos, mas decisão é manual por risco regulatório

entrega contínua

pseudo-maturidade

CI fraco

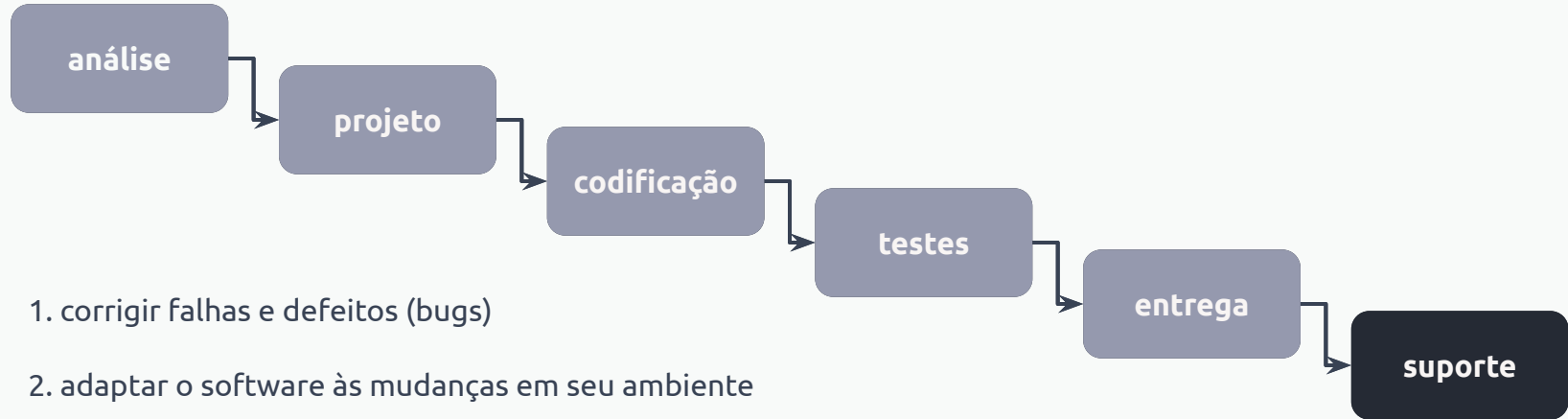
entrega contínua

Evolução do Desenvolvimento de Software

definindo a maturidade

- 1) integra frequentemente com build/testes confiáveis?
 - └ NÃO → prioridade: integração contínua
 - └ SIM →
- 2) está sempre pronto para produção (artefato+staging+checklist)?
 - └ NÃO → prioridade: entrega contínua
 - └ SIM →
- 3) operação confiável (observabilidade+rollback+baixa falha)?
 - └ NÃO → manter entrega contínua e fortalecer operação
 - └ SIM → piloto de implantação contínua

Evolução do Desenvolvimento de Software



1. corrigir falhas e defeitos (bugs)
2. adaptar o software às mudanças em seu ambiente
3. aprimorar o software com base nas solicitações das partes interessadas
4. reprojetar o software para obter melhor funcionalidade e desempenho

Foco da Manutenção

1. manutenção corretiva
2. manutenção adaptativa
3. manutenção perfectiva
4. manutenção preventiva



Teoria Unificada para a Evolução do Software

(Lehman et al., 1997)

Lei da mudança contínua (1974). Um sistema implementado em um contexto de computação do mundo real (evolui ao longo do tempo) deve ser continuamente adaptado, caso contrário, ele se tornará progressivamente menos satisfatório.

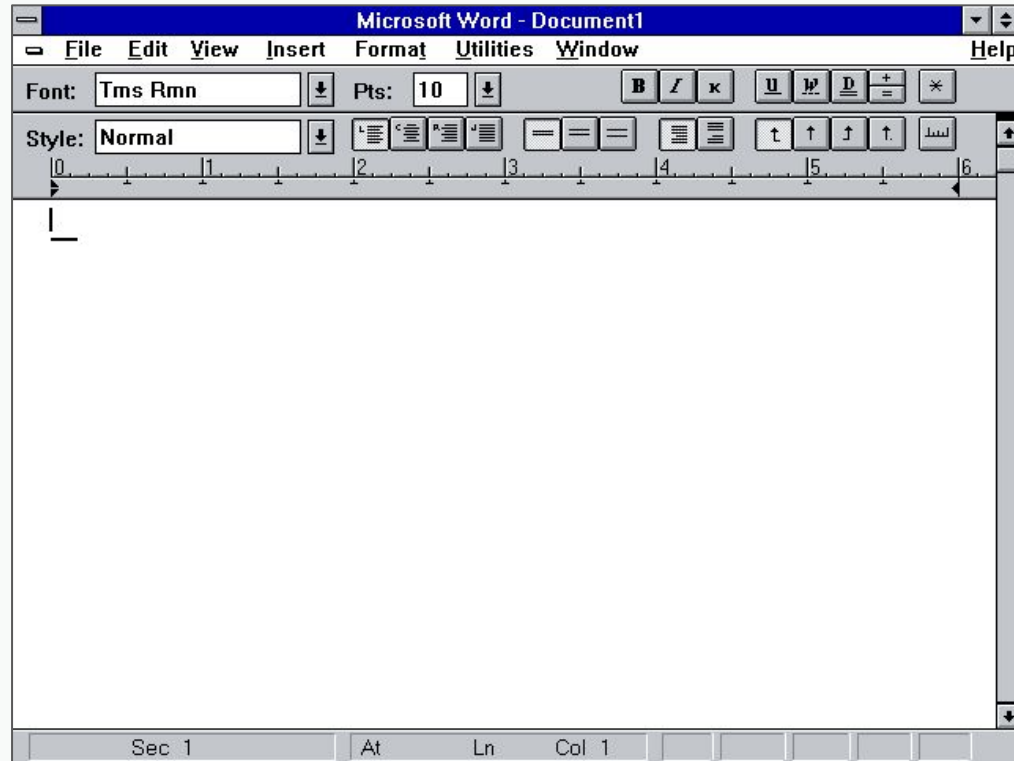
Lei da complexidade crescente (1974). À medida que um sistema evolui, sua complexidade aumenta, a menos que seja feito trabalho para mantê-la ou reduzi-la.

Lei da conservação da familiaridade (1980). À medida que um sistema evolui, todos os associados a ele devem manter o conhecimento de seu conteúdo e comportamento para uma evolução satisfatória. O crescimento excessivo diminui esse conhecimento. Portanto, o crescimento incremental médio permanece invariável à medida que o sistema evolui.

Lei do crescimento contínuo (1980). O conteúdo funcional dos sistemas deve aumentar continuamente para manter a satisfação do usuário ao longo de sua vida útil.

Lei da qualidade decrescente (1996). A qualidade dos sistemas parecerá declinar se não forem rigorosamente mantidos e adaptados às mudanças do ambiente operacional.

Teoria Unificada para a Evolução do Software





**INSTITUTO
FEDERAL**
Santa Catarina

Evolução de Software

Engenharia de Software II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br

